

COMBINATIONAL LOGIC DESIGN

Module.2

CDA 3103C

Spring 2026

Mesut Ozdag, Ph.D.

Logic functions and Boolean algebra

- Any logic function that can be expressed as a truth table can be written as an expression in Boolean algebra using the operators: ', +, and •

X	Y	$X \cdot Y$
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	X'	$X' \cdot Y$
0	0	1	0
0	1	1	1
1	0	0	0
1	1	0	0

X	Y	X'	Y'	$X \cdot Y$	$X' \cdot Y'$	$(X \cdot Y) + (X' \cdot Y')$
0	0	1	1	0	1	1
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	1	0	0	1	0	1

$$(X \cdot Y) + (X' \cdot Y') = \overline{X \oplus Y}$$

X, Y are Boolean algebra variables

Boolean expression that is true when the variables X and Y have the same value and false, otherwise

Logic functions and Boolean algebra

- D is true if at least one input is true
- E is true if exactly two inputs are true
- F is true only if all three inputs are true.

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

- $D = A+B+C$
- $F = A*B*C$
- $E = ((A * B) + (A * C) + (B * C)) * (A * B * C)'$
- $E = (A * B * C') + (A * C * B') + (B * C * A')$

Proving theorems with Perfect Induction

- Using perfect induction (complete truth table):
 - e.g., de Morgan's:

$(X + Y)' = X' \cdot Y'$
NOR is equivalent to AND
with inputs complemented

X	Y	X'	Y'	$(X + Y)'$	$X' \cdot Y'$
0	0	1	1	1	1
0	1	1	0	0	0
1	0	0	1	0	0
1	1	0	0	0	0

$(X \cdot Y)' = X' + Y'$
NAND is equivalent to OR
with inputs complemented

X	Y	X'	Y'	$(X \cdot Y)'$	$X' + Y'$
0	0	1	1	1	1
0	1	1	0	1	1
1	0	0	1	1	1
1	1	0	0	0	0

Proving theorems with Perfect Induction

- $E = ((A * B) + (A * C) + (B * C)) * (A * B * C)'$
- $E = (A * B * C') + (A * C * B') + (B * C * A')$

Inputs			E		
A	B	C	E	$((A * B) + (A * C) + (B * C)) * (A * B * C)'$	$(A * B * C') + (A * C * B') + (B * C * A')$
0	0	0	0	0	0
0	0	1	0	0	0
0	1	0	0	0	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	1	1	1
1	1	0	1	1	1
1	1	1	0	0	0

Proving theorems with Rewriting

- Using the axioms of Boolean algebra:

- e.g., prove the uniting theorem:

	$X \cdot Y + X \cdot Y' = X$
Distributive law	$X \cdot Y + X \cdot Y' = X \cdot (Y + Y')$
Inverse law	$X \cdot (Y + Y') = X \cdot (1)$
Identity law	$X \cdot (1) = X \checkmark$

- e.g., prove the absorption theorem:

$$X + X \cdot Y = X$$

Identity law	$X + X \cdot Y$	$= X \cdot 1 + X \cdot Y$
Distributive law	$X \cdot 1 + X \cdot Y$	$= X \cdot (1 + Y)$
Null law	$X \cdot (1 + Y)$	$= X \cdot (1)$
Identity law	$X \cdot (1)$	$= X \checkmark$

Proving theorems with Rewriting

- $E = ((A * B) + (A * C) + (B * C)) * (A * B * C)'$
- $E = (A * B * C') + (A * C * B') + (B * C * A')$

$$\begin{aligned} & ((A * B) + (A * C) + (B * C)) * (A * B * C)' \\ &= ((A * B) + (A * C) + (B * C)) * (A' + B' + C') \\ & \quad \text{DeMorgan's law} \end{aligned}$$

$$\begin{aligned} &= (A' + B' + C')(A * B) + (A' + B' + C')(A * C) + (A' + B' + C')(B * C) \\ & \quad \text{Distributive law} \end{aligned}$$

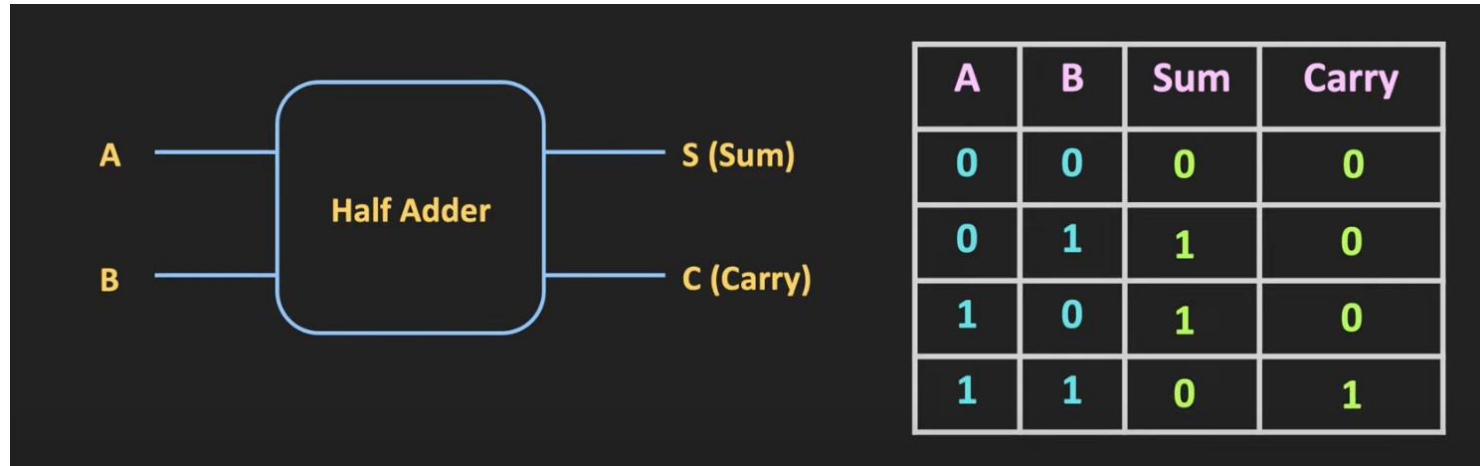
$$\begin{aligned} &= (A * B * A') + (A * B * B') + (A * B * C') + (A * C * A') + (A * C * B') + (A * C * C') + \\ & \quad (B * C * A') + (B * C * B') + (B * C * C') \\ & \quad \text{Distributive law} \end{aligned}$$

$$\begin{aligned} &= (0 * B) + (0 * A) + (A * B * C') + (0 * C) + (A * C * B') + (A * 0) + (B * C * A') + (C * 0) + (B * 0) \\ & \quad \text{Inverse law} \end{aligned}$$

$$\begin{aligned} &= 0 + 0 + (A * B * C') + 0 + (A * C * B') + 0 + (B * C * A') + 0 + 0 \\ & \quad \text{Null law} \end{aligned}$$

$$\begin{aligned} &= (A \cdot B \cdot C') + (A \cdot C \cdot B') + (B \cdot C \cdot A') \\ & \quad \text{Identity law} \end{aligned}$$

Half Adder



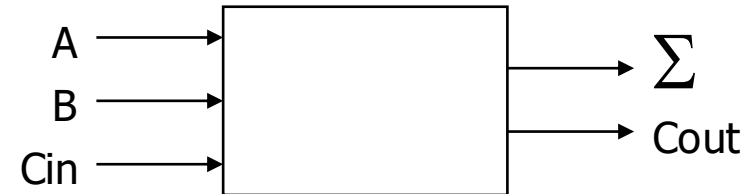
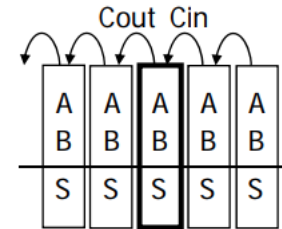
$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

Adder, Part 1

- 1-bit binary adder
 - inputs: A, B, Carry-in
 - outputs: Sum, Carry-out

A	B	Cin	Σ	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$$\Sigma = A' B' Cin + A' B Cin' + A B' Cin' + A B Cin$$

$$Cout = A' B Cin + A B' Cin + A B Cin' + A B Cin$$

Apply the theorems to simplify expressions

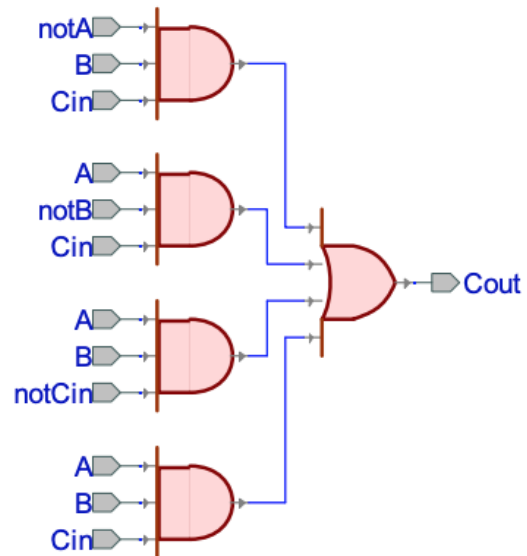
- The theorems of Boolean algebra can simplify Boolean expressions
 - The Cout function is used as an example here, but the same rules apply to any function.

$$\begin{aligned}\text{Cout} &= A' B \text{Cin} + A B' \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Idempotency}) &= A' B \text{Cin} + A B' \text{Cin} + A B \text{Cin}' + A B \text{Cin} + A B \text{Cin} \\ (\text{Commutative}) &= A' B \text{Cin} + A B \text{Cin} + A B' \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Distributive}) &= (A' + A) B \text{Cin} + A B' \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Inverse}) &= (1) B \text{Cin} + A B' \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Idempotency}) &= B \text{Cin} + A B' \text{Cin} + A B \text{Cin}' + A B \text{Cin} + A B \text{Cin} \\ (\text{Commutative}) &= B \text{Cin} + A B' \text{Cin} + A B \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Distributive}) &= B \text{Cin} + A (B' + B) \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Inverse}) &= B \text{Cin} + A (1) \text{Cin} + A B \text{Cin}' + A B \text{Cin} \\ (\text{Distributive}) &= B \text{Cin} + A \text{Cin} + A B (\text{Cin}' + \text{Cin}) \\ (\text{Inverse}) &= B \text{Cin} + A \text{Cin} + A B (1) \\ &= B \text{Cin} + A \text{Cin} + A B\end{aligned}$$

Full adder: Carry-out

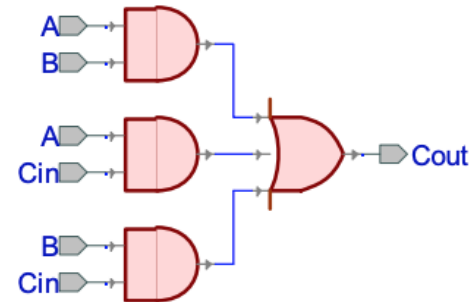
Before Boolean minimization

$$\begin{aligned} \text{Cout} = & A'BC_{in} + AB'C_{in} \\ & + ABC_{in}' + ABC_{in} \end{aligned}$$



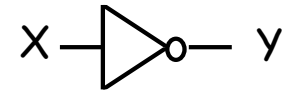
After Boolean minimization

$$\text{Cout} = BC_{in} + AC_{in} + AB$$



Logic Gates

- NOT X' $\bar{X}$ $\sim X$



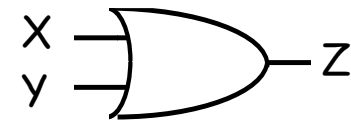
X	Y
0	1
1	0

- AND $X \cdot Y$ $X \wedge Y$



X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

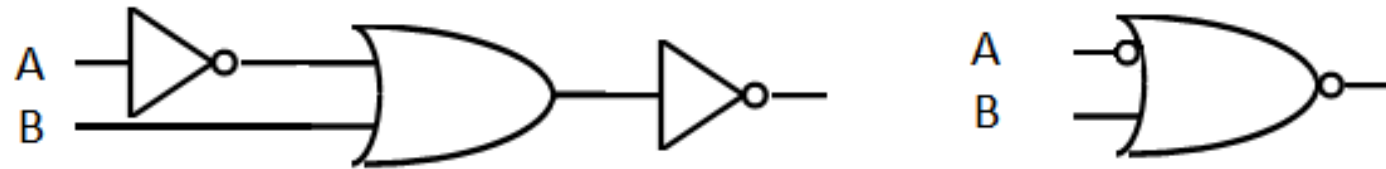
- OR $X + Y$ $X \vee Y$



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	1

Logic Gates and Inverters

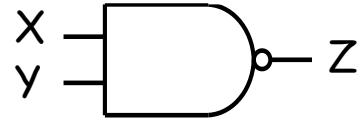
- Rather than draw inverters explicitly, a common practice is to add “bubbles” to the inputs or outputs of a gate to cause the logic value on that input line or output line to be inverted.



$$(A' + B)'$$

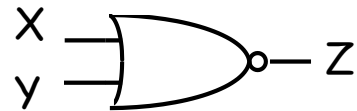
Logic Gates

- NAND



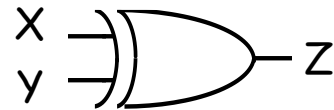
X	Y	Z
0	0	1
0	1	1
1	0	1
1	1	0

- NOR



X	Y	Z
0	0	1
0	1	0
1	0	0
1	1	0

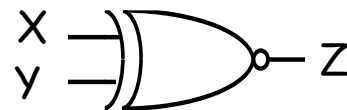
- XOR
 $X \oplus Y$



X	Y	Z
0	0	0
0	1	1
1	0	1
1	1	0

$X \text{ xor } Y = X Y' + X' Y$
X or Y but not both
("inequality", "difference")

- XNOR
 $\overline{X \oplus Y}$



X	Y	Z
0	0	1
0	1	0
1	0	0
1	1	1

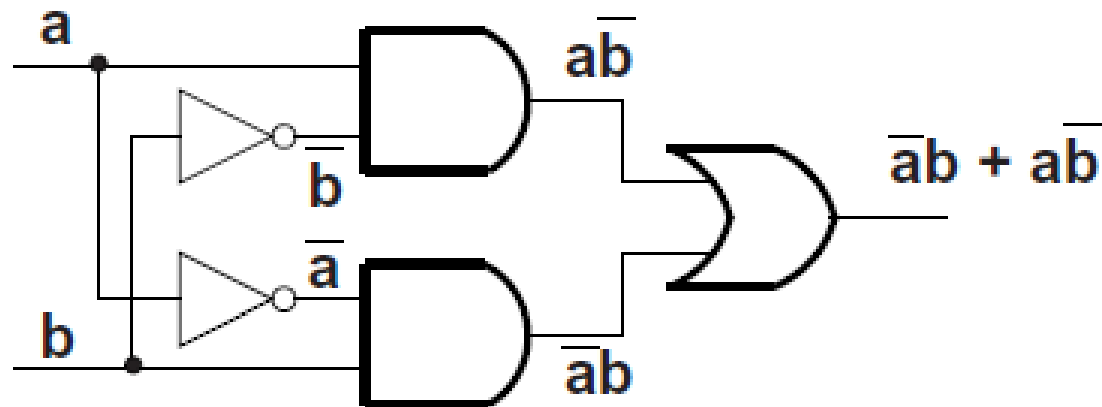
$X \text{ xnor } Y = X Y + X' Y'$
X and Y are the same
("equality", "coincidence")

Adder, Part 2

- Logic gates for Sum (shown without Cin)

A	B	Sum
0	0	0
0	1	1
1	0	1
1	1	0

$$\text{Sum} = A' B + A B'$$

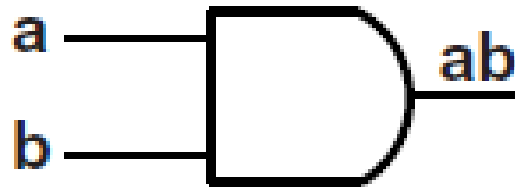


Adder, Part 2

- Logic gates for Cout (shown without Cin)

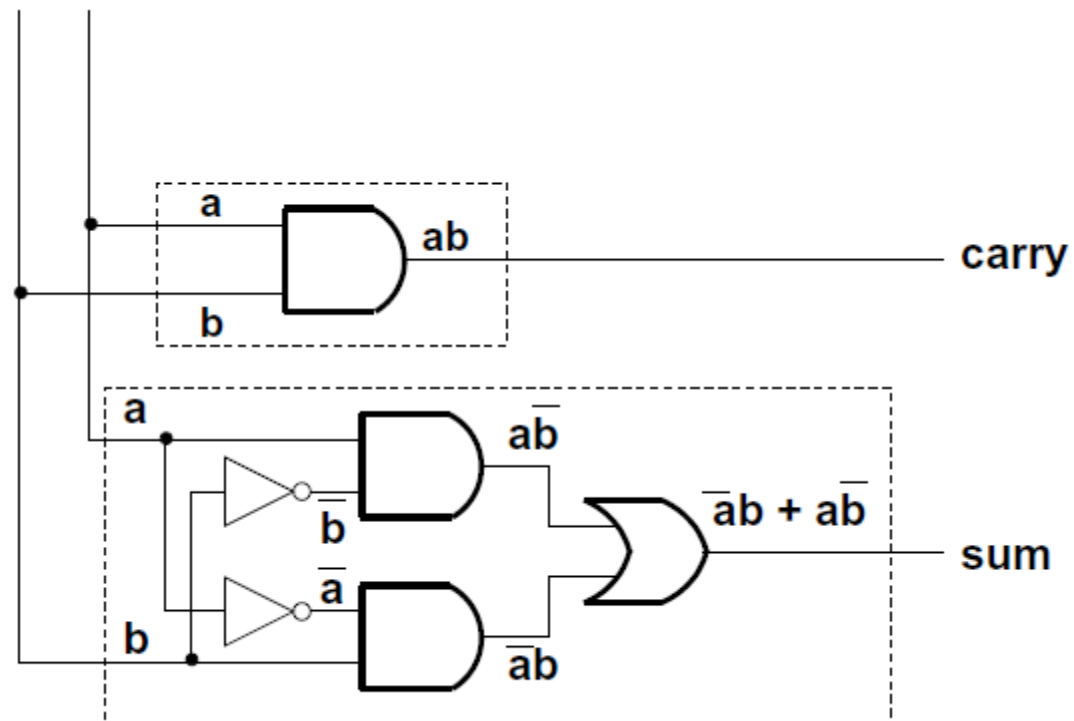
A	B	Cout
0	0	0
1	0	0
0	1	0
1	1	1

$$Cout = A B$$



Adder, Part 2

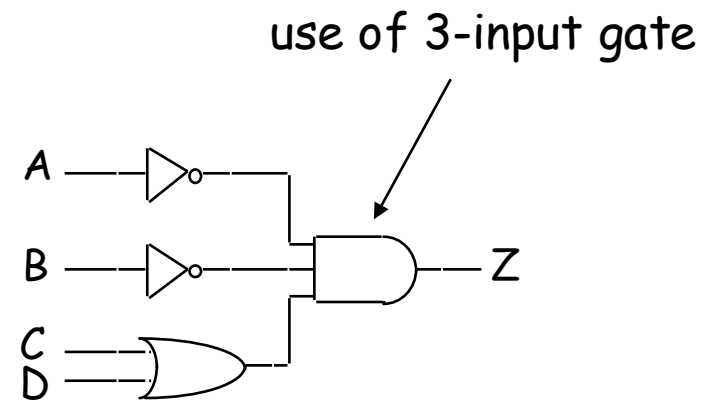
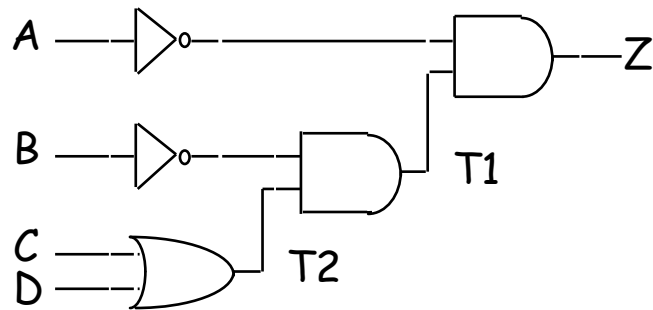
- Logic gates for Sum and Cout (shown without Cin)



Logic Gates

- More than one way to map expressions to gates

- e.g.,
$$Z = A' \cdot B' \cdot (C + D) = \underbrace{(A' \cdot (B' \cdot \underbrace{(C + D)}_{T2}))}_{T1}$$





MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU

